

QBiC-SELEX Batched Statistics Optimization

Overview

This document explains the batched matrix optimization applied to `qbic_predict_faster.py`, which replaces the per-variant loop in `--compute-stats` with vectorized matrix operations. The result is mathematically identical but orders of magnitude faster for large-scale runs (10M+ variants x 1000+ models).

1. What the Original Code Does

For each variant, `qbic_predict.py` computes a t-score and p-value using a linear model and its covariance matrix.

Per-variant computation (original)

```
def compute_p_value_cpu(alt_kmers, ref_kmers, partial_dict, cov_matrix):
    c = convert_kmers_to_c_vectorized(alt_kmers, ref_kmers, partial_dict) # (d,)
    lm_values = np.array(list(partial_dict.values()), dtype=np.float32) # (d,)

    estimate = np.dot(c.T, lm_values) # scalar: c^T @ beta
    se_estimate = np.sqrt(np.dot(np.dot(c.T, cov_matrix), c)) # scalar: sqrt(c^T @ Sigma @ c)
    t_score = estimate / se_estimate
    p_value = 2 * stats.norm.sf(np.abs(t_score))

    return float(t_score), float(p_value)
```

What each variable means

Symbol	Code variable	Shape	Meaning
c	c	(d,) where d=8192	Coefficient vector: alt k-mer counts minus ref k-mer counts, mapped to the model's partial dictionary
beta	lm_values	(d,)	Linear model coefficients (the model's learned weights for each k-mer)
Sigma	cov_matrix	(d, d)	Covariance matrix of the model coefficients
estimate	estimate	scalar	Predicted binding effect: $c^T @ \beta$

Symbol	Code variable	Shape	Meaning
se	se_estimate	scalar	Standard error of the estimate: $\text{sqrt}(c^T @ \text{Sigma} @ c)$
t	t_score	scalar	t-statistic: $\text{estimate} / \text{se}$
p	p_value	scalar	Two-tailed p-value from the normal distribution

The loop structure (original)

This per-variant function is called inside a Python loop:

```
for each variant (10 million times):
    c = build_coefficient_vector(alt_kmers, ref_kmers) # shape: (8192,)
    estimate = c^T @ beta # dot product
    se = sqrt(c^T @ Sigma @ c) # quadratic form
    t = estimate / se
    p = 2 * Phi(-|t|)
```

For 10M variants x 1000 models, this loop executes **10 billion times**.

2. The Core Idea: Vectorize Across Variants

Step 1: Stack individual vectors into a matrix

The original builds one c vector (length 8192) per variant. If you have N variants, that is N separate vectors. The key insight: **stack them into a 2D matrix**.

Original (per-variant):	Batched (all-at-once):
$c_0 = [0, 1, -1, 0, \dots]$	$C = [c_0]$ shape: (N, 8192)
$c_1 = [0, 0, 1, 0, \dots]$	$[c_1]$
$c_2 = [1, 0, 0, -1, \dots]$	$[c_2]$
$\dots$	$[\dots]$
$c_N = [0, -1, 0, 0, \dots]$	$[c_N]$

This is what `_build_c_matrix()` does in the faster version. It uses the exact same k-mer-to-index logic as the original `convert_kmers_to_c_vectorized()`, but fills rows of a 2D array instead of returning individual 1D vectors.

Step 2: Replace the loop with matrix operations

Estimates (binding effect predictions) The original does `np.dot(c.T, lm_values)` per variant in a Python loop:

```

# Original: N separate dot products in a Python loop
for i in range(N):
    estimates[i] = c_i @ beta      # one at a time

# Batched: ONE matrix-vector multiply
estimates = C @ beta              # (N, 8192) @ (8192,) = (N,)

```

This works because row i of $C @ \text{beta}$ is exactly $C[i,:] @ \text{beta} = c_i @ \text{beta}$.

Standard errors (the expensive part) The original computes a quadratic form per variant:

```

# Original: N separate quadratic forms in a Python loop
for i in range(N):
    se[i] = sqrt(c_i^T @ Sigma @ c_i)  # two matmuls per variant

```

The batched version replaces this with:

```

# Batched: two matrix operations for ALL variants
C_cov = C @ Sigma                      # (N, 8192) @ (8192, 8192) = (N, 8192)
se = np.sqrt(np.sum(C_cov * C, axis=1)) # element-wise multiply + row sum = (N,)

```

```

# Original: one scipy call per variant in a loop
for i in range(N):
    p_values[i] = 2 * stats.norm.sf(abs(t_scores[i]))

# Batched: scipy handles arrays natively
p_values = 2.0 * stats.norm.sf(np.abs(t_scores)) # all N at once

```

P-values

3. Mathematical Proof: Why $\text{row_sum}((C @ \text{Sigma}) * C)$ equals each $c_i^T @ \text{Sigma} @ c_i$

This is the critical identity that makes the optimization valid.

The claim

$\text{np.sum}((C @ \text{Sigma}) * C, \text{axis}=1)[i] == c_i^T @ \text{Sigma} @ c_i$ for all i

The proof

Write out what each expression computes for row i :

Left side (batched computation):

Step 1: $(C @ \text{Sigma})[i, j] = \sum_k C[i, k] * \text{Sigma}[k, j]$

Step 2: $((C @ \text{Sigma}) * C)[i, j] = (C @ \text{Sigma})[i, j] * C[i, j]$

```

Step 3: row_sum[i] = sum_j ((C @ Sigma) * C)[i, j]
          = sum_j (sum_k C[i,k] * Sigma[k,j]) * C[i,j]
          = sum_j sum_k C[i,k] * Sigma[k,j] * C[i,j]

```

Right side (original per-variant computation):

```
c_i^T @ Sigma @ c_i = sum_j sum_k c_i[k] * Sigma[k,j] * c_i[j]
```

Since $C[i,k] = c_i[k]$ and $C[i,j] = c_i[j]$ by construction, these are **identical**.

Geometric interpretation

The full expression $C @ \text{Sigma} @ C^T$ would be an $(N \times N)$ matrix whose diagonal entries are exactly $c_i^T @ \text{Sigma} @ c_i$. But we only need the diagonal, not the full matrix. The `row_sum((C @ Sigma) * C)` trick computes **only the diagonal** without ever building the $(N \times N)$ matrix, which would be impossibly large (10M x 10M).

This identity is sometimes called the “Hadamard product trace trick” or “diagonal extraction trick.”

4. Why Batching is So Much Faster

The speedup comes from three sources:

4.1 BLAS optimization

NumPy’s matrix multiply (`@`) delegates to BLAS (Basic Linear Algebra Subprograms), which is written in hand-tuned C/Fortran. BLAS uses:

- **SIMD instructions** (AVX-512): processes 16 float32 values in a single CPU instruction
- **Cache blocking**: arranges computation to maximize L1/L2/L3 cache hits
- **Multi-threading**: uses all CPU cores automatically via OpenMP

A single $(20000, 8192) @ (8192, 8192)$ matmul gives BLAS a huge block of work to optimize. The per-variant `np.dot(c, np.dot(cov, c))` calls are too small (just 8192 elements) for BLAS to optimize efficiently. The overhead of each call (function dispatch, memory setup) dominates the actual computation.

4.2 Python loop elimination

Each iteration through a Python `for` loop costs ~100 nanoseconds of interpreter overhead (bytecode dispatch, reference counting, etc.). For 10M variants, that is ~1 second just in loop bookkeeping, before any math happens.

The batched version loops over ~500 chunks (10M / 20K) instead of 10M variants. That is a 20,000x reduction in Python overhead.

4.3 GPU utilization (for the GPU path)

GPUs have thousands of cores (e.g., 10,496 on an A100) but need large parallel workloads to saturate them. A single 8192-element dot product uses less than 1% of GPU capacity, meaning 99% of the GPU sits idle while processing one variant.

A (100K, 8192) @ (8192, 8192) matmul provides ~800 million operations, enough to utilize nearly 100% of GPU capacity.

Speed comparison

Method	How it works	Estimated time (10M variants, 1 model)
Original CPU (1 core)	10M individual dot products in Python loop	~30-60 min
Original CPU (32 cores)	10M tasks in ProcessPoolExecutor + IPC overhead	~2-5 min
Original GPU (per-variant)	10M individual GPU kernel launches	~5-15 min
Batched CPU	~500 BLAS matmuls of (20K, 8192) @ (8192, 8192)	~1-3 min
Batched GPU	~100 GPU matmuls of (100K, 8192) @ (8192, 8192)	~5-15 sec

5. Chunking Strategy

We cannot build a (10M, 8192) matrix. That would be 80 GB in float32. Instead, we process in chunks:

10M variants, chunk_size = 20,000 -> 500 chunks

For each chunk:

```

C      = (20K, 8192)  int8    -> ~160 MB
C_f    = (20K, 8192)  float32 -> ~640 MB
C @ cov = (20K, 8192)  float32 -> ~640 MB
Total per chunk: ~1.4 GB (fits comfortably in RAM)

```

For the GPU path, chunk_size defaults to 100,000:

```

C_gpu   = (100K, 8192) float32 -> ~3.2 GB
C @ cov = (100K, 8192) float32 -> ~3.2 GB
cov_gpu = (8192, 8192) float32 -> ~256 MB (loaded once)
Total GPU memory: ~6.7 GB (fits on 8GB+ GPU)

```

The chunk size can be tuned based on available memory.

Implementation

```
for start in range(0, n_variants, chunk_size):
    end = min(start + chunk_size, n_variants)

    C, bad_rows = _build_c_matrix(alt_list, ref_list, partial_dict, start, end)
    C_f = C.astype(np.float32)

    estimates = C_f @ lm_values                # (chunk_n,)
    C_cov = C_f @ cov                        # (chunk_n, d)
    se_all = np.sqrt(np.sum(C_cov * C_f, axis=1)) # (chunk_n,)

    t_chunk = estimates / se_all
    p_chunk = 2.0 * stats.norm.sf(np.abs(t_chunk))

    _apply_edge_cases(t_chunk, p_chunk, estimates, se_all, bad_rows)

    t_scores[start:end] = t_chunk
    p_values[start:end] = p_chunk
```

Results from each chunk are written into pre-allocated output arrays at the correct slice positions. No results are lost or mixed between chunks.

6. Edge Case Handling

The original code has three edge cases. The batched version replicates all of them.

Edge case 1: estimate == 0

Original: if estimate == 0: return 0.0, 1.0 (early return)

Batched: The matrix math runs on all rows (including zero-estimate ones). After computation, a boolean mask overrides those entries:

```
zero_mask = (estimates == 0)
t_chunk[zero_mask] = 0.0
p_chunk[zero_mask] = 1.0
```

Edge case 2: Invalid standard error (se <= 0, NaN, or Inf)

Original: Raises an exception, caught by the caller, returns (nan, nan).

Batched: A zero row in C produces se=0, and division by zero produces inf or nan in numpy (suppressed with np.errstate). Then:

```
bad_se = (se_all <= 0) | np.isnan(se_all) | np.isinf(se_all)
t_chunk[bad_se & ~zero_mask] = np.nan
p_chunk[bad_se & ~zero_mask] = np.nan
```

Edge case 3: Exception during k-mer processing

Original: Per-variant try/except catches any error, returns (nan, nan).

Batched: During `_build_c_matrix()`, each row is built inside a try/except. If a row fails, it stays as all zeros and its index is recorded in `bad_rows`. After the matrix math:

```
for bi in bad_rows:
    t_chunk[bi] = np.nan
    p_chunk[bi] = np.nan
```

Why one bad variant cannot affect other rows

Matrix multiplication is row-independent: $(C @ \Sigma)[i, :]$ depends ONLY on $C[i, :]$. A NaN or zero in row i never propagates to row j . Similarly, `np.sum(..., axis=1)` sums within each row independently, and `estimates / se_all` is element-wise division.

This “compute-then-mask” strategy is faster than checking conditions inside the loop because: - The matmul runs without branching (no if/else per row) - Boolean masking is itself a vectorized numpy operation - The vast majority of variants are not edge cases

7. Floating Point Differences

The batched version produces results that differ from the original by a tiny amount. This is expected and scientifically insignificant.

Why the difference exists

BLAS matrix multiplication uses a different summation order than a manual dot product loop. For example, the original `np.dot(c.T, np.dot(cov, c))` sums 8192 terms in a specific sequential order. The batched `C @ cov` uses optimized BLAS routines that may use:

- Fused multiply-add (FMA) instructions
- Different accumulation order for cache efficiency
- SIMD-width-aligned partial sums

Floating point addition is not associative: $(a + b) + c \neq a + (b + c)$ in general due to rounding. Different summation orders produce different rounding errors.

How large is the difference?

From validation on the 199-variant example dataset:

Metric	z_score	p_value
Max absolute diff	7.63e-06	1.38e-06
Max relative diff	3.15e-05	4.22e-05
Median relative diff	1.19e-07	1.01e-08
99th percentile relative diff	6.62e-06	-

These are well within float32 precision ($\sim 1e-7$ machine epsilon, $\sim 1e-5$ relative error for accumulated operations). The `z_scores` agree to at least 4 significant digits. No scientifically meaningful difference.

Verification

```
assert np.allclose(z_original, z_batched, rtol=1e-4) # passes
assert np.allclose(p_original, p_batched, rtol=1e-4) # passes
```

8. Visual Summary

ORIGINAL (per-variant loop):	BATCHED (matrix ops):
variant 0: <code>c_0 -> dot -> se -> t -> p</code>	Build C matrix (all variants at once)
variant 1: <code>c_1 -> dot -> se -> t -> p</code>	
variant 2: <code>c_2 -> dot -> se -> t -> p</code>	<code>estimates = C @ beta</code> (one BLAS call)
... (10M Python iterations)	<code>C_cov = C @ Sigma</code> (one BLAS call)
variant N: <code>c_N -> dot -> se -> t -> p</code>	<code>se = sqrt(rowsum(C_cov*C))</code> (vectorized)
	<code>t = estimates / se</code> (element-wise)
Each iteration:	<code>p = 2 * norm.sf(t)</code> (vectorized)
- Python loop overhead (~ 100 ns)	<code>apply_edge_cases(masks)</code> (vectorized)
- Tiny BLAS call (8192 elements)	
- scipy call (1 scalar)	Total: ~ 3 BLAS calls + 1 scipy call
	per chunk of 20K variants

9. Key Takeaway

The fundamental principle: **give optimized C/Fortran libraries (BLAS, LAPACK) large blocks of work** instead of calling them millions of times with tiny inputs.

This is the single most common performance pattern in scientific Python. Whenever you see a Python loop calling numpy on small arrays, ask: “Can I stack these into a matrix and do one big operation instead?”

Files

File	Description
<code>qbic_predict.py</code>	Original implementation with per-variant loop
<code>qbic_predict_faster.py</code>	Optimized implementation with batched matrix operations

The faster version is a drop-in replacement with the identical CLI interface.